Summary Introduction to Python

Introduction to python (Universiteit van Amsterdam)

SUMMARY: INTRODUCTION TO PYTHON

1. **Practical information**
2. **Python basics**
3. **Lists**
4. **Functions**
5. **Numpy**
6. **Dictionaries**
7. **Loops**

1. Practical information

In the console, you are always in some part of the file structure: pwd
Run program: **python3 1.py (tab)**
Correct code? **python3 t(tab) 1**

Pythoncheatsheets.com

2. Python basics

float = real numbers
int = integer (heel getal)
str = text: double or single quotes
bool = True/False

```
print("I started with $" + savings + " and now have $" + result + ". Awesome!")
```

→ Will not work as savings and result are integers/floats: convert types of variables to strings (text)

```
# Fix the printout
print("I started with $" + str(savings) + " and now have $" + str(result) + ". Awesome!")

# Definition of pi_string
pi_string = "3.1415926"

# Convert pi_string into float: pi_float
pi_float = float(pi_string)
```

type(bmi) → gives the type of a variable
How code behaves depends on the types of variables

3. Lists

Python list: [a, b, c] – name of collection of values that can be different variable types
a, b, c are data points

```
# area variables (in square meters)
hall = 11.25
kit = 18.0
liv = 20.0
bed = 10.75
bath = 9.50

# Create list areas
```

```
areas = [hall, kit, liv, bed, bath]
```

List **slicing** – select multiple elements within a list → create new list
fam[start : end] where start is included and end is excluded
fam[1:4] gives elements one, two and three
fam[:4] gives first, second, third and fourth
fam[5:] selects all elements up to and including the last element of the list, gives 5th to last

```
# Alternative slicing to create downstairs
downstairs = areas[:6]

# Alternative slicing to create upstairs
upstairs = areas[6:]
```

Adding and **removing** elements:
fam + [“me”, 1.79]
fam_ext = fam + [“me”, 1.79]
del(fam[2]) → variables will move over!

```
# Add poolhouse data to areas, new list is areas_1
areas_1 = areas + ["poolhouse", 24.5]
print(areas_1)
```

List **manipulation:**
Changing list elements:
fam[7] = 1.86
fam[0:2] = [“lisa”, 1.74]

Copying lists:
areas
areas_copy = areas → changes in areas_copy also happen to areas
areas_copy =. list(areas) → changes in areas_copy do not happen to areas

When copying x to y with the equal sign, you copy the reference to the list, not the actual values themselves. Update is visible for both variables.
If you want to create a list y that points to a new list with the same values as x, you will need something other than the equal function:
y = list(x)
y = x[:]

List **methods**
fam (list) has the method ‘index’. To call the method: fam.index(“mom”) → and the method ‘count’. To call the count: fam.count(1.73) → 1 (times it occurs in the list)
index()
count()
append() → adds element to list
remove() → removes first element of a list that matches the input
reverse() → reverses the order of the elements in the list

```
# Print out the index of the element 20.0
print(areas.index(20.0))
# Print out how often 9.50 appears in areas
print(areas.count(9.50))
# Use append twice to add poolhouse and garage size
areas.append(24.5)
areas.append(15.45)

# Reverse the orders of the elements in areas
areas.reverse()
```

4. FUNCTIONS

max()
round(number[, ndigits])
len()
sorted()

Help on specific function:
- help(round)
- ?max

Methods: functions that belong to specific python objects
Depending on the type of the object, methods behave differently. Some methods can change the object they are called on.

- str methods
  sister.capitalize() → capitalizes the first letter
  sister.replace("z", "sa")

- float methods
  bit_length()
  conjugate()

- list methods
  help(str) / help(float) etc. gives possible methods

place is new variable
place.upper → all letters capitalized
place.count('o') → counts number of 'o's in variable place (="poolhouse")

5. NUMPY

Python cannot do calculations on lists → solution: Numeric Python (Numpy)

Installation - Terminal: pip3 install numpy

```
import numpy as np
```

Creating a numpy array function:

```
array([1.73, 1.69, 1.71])
np_height = np.array(height)

np_weight = np.array(weight)

bmi = np_weight / np_height ** 2
```

Remarks:

- Numpy arrays can only one type
- Numpy array is basically its own type

```
python_list = [1, 2, 3]
numpy_array = np.array([1, 2, 3])
```

python_list + python_lits → [1, 2, 3, 1, 2, 3]
numpy_array + numpy_array → array([2, 4, 6])

Numpy **subsetting**:

```
bmi[1]
```

bmi > 23 → array([False, False, False, True, False], dtype=bool)
bmi[bmi > 23] → array([24.747])

Numpy side effects:

1) Numpy arrays cannot contain elements with different types. **Type coercion**: elements' types are changed to end up with a homogeneous list
2) Typical arithmetic operators (+, -, * and /) have a different meaning for regular Python lists and numpy arrays

```
# Create list baseball
baseball = [180, 215, 210, 210, 188, 176, 209, 200]

# Import the numpy package as np
import numpy as np

# Create a numpy array from baseball: np_baseball
np_baseball = np.array(baseball)
```

Subsetting (using the square bracket notation on lists or arrays) works exactly the same:

```
# Print out the weight at index 50
print(np_weight_lb[50])

# Print out sub-array of np_height_in: index 100 up to and including index 110
print(np_height_in[100:111])
```

2D NUMPY ARRAYS

Subsetting:
np_2d = np.array([[1.73, 1.68, 1.71],
[65.4, 59.2, 63.6]])

np_2d.shape → (2, 5) # 2 rows, 5 columns

np_2d[0][2] → [row][element]
np_2d[0,2] → [row, element]
np_2d[:, 1:3] → all rows, elements 1 to 2

```
# Import numpy package
import numpy as np

# Create np_baseball (2 cols)
np_baseball = np.array(baseball)

# Print out the 50th row of np_baseball
print(np_baseball[49,:])

# Select the entire second column of np_baseball: np_weight_lb
np_weight_lb = np_baseball[:, 1]

# Print out height of 124th player
print(np_baseball[123, 0])
```

6. DICTIONARIES

Dictionaries
pop = [30.55, 2.77, 39.21]
countries = [“afghanistan”, “albania”, “algeria”]

Want to know the population of Albania?

ind_albania = countries.index(“albania”)
ind_albania gives 1

pop[ind_albania] or pop[1] gives 2.77

Not convenient – Connect country to its population without using index → Dictionary

Dictionary:
world = {“afghanistan”:30.55, “albania”:2.77, “algeria”:39.21}
world[“albania”] gives 2.77

7. LOOPS

For loop – “for each variable in a sequence, execute expression”

List:

```
fam = [1.73, 1.68, 1.71, 1.89]
for height in fam:
        print(height)
```

Using a for loop to iterate over a list only gives you access to every list element in each run, one after the other. If you also want to access the index information, so where the list element you're iterating over is located, you can use enumerate().

```
for index, height in enumerate(fam):
        print(“index” + str(index) + “: “ + str(height))
```

```
# areas list
areas = [11.25, 18.0, 20.0, 10.75, 9.50]

# Change for loop to use enumerate() and update print()
for index, a in enumerate(areas):
   print("room" + str(index) + ":" + str(a))
```

```
# house list of lists
house = [["hallway", 11.25],
        ["kitchen", 18.0],
        ["living room", 20.0],
        ["bedroom", 10.75],
        ["bathroom", 9.50]]

# Build a for loop from scratch
for x in house:
   print("the " + x[0] + " is " + str(x[1]) + " sqm")
```

for loop in dictionaries with keys and values

```
# Definition of dictionary
europe = {'spain':'madrid', 'france':'paris', 'germany':'berlin',
        'norway':'oslo', 'italy':'rome', 'poland':'warsaw', 'austria':'vienna' }

# Iterate over europe
for country, value in europe.items():
   print("the capital of " + country + " is " + str(value))
```

for loop for n-dimensional arrays:

1 dimensional:

```
for x in my_array :
    ...
```

Multi dimensional:

```
for x in np.nditer(my_array) :
    ...
```

```
# Import numpy as np
import numpy as np

# For loop over np_height
for height in np_height:
    print(str(height) + " inches")

# For loop over np_baseball
for x in np.nditer(np_baseball):
    print(x)
```

For loop for Pandas DataFrame:

Iterating over the rows: iterrows
Used in a for loop, every observation is iterated over and on every iteration the row label and actual row contents are available:

```
for lab, row in brics.iterrows() :
                print(lab)
                print(row)
```

Install conda
run file: pyhon pythonfilename
conda list
conda install pandas